Aula 09 – MIPS Pipeline: Solucionando Hazards de Controle

Prof. João Fernando Mari
joaofmari.github.io

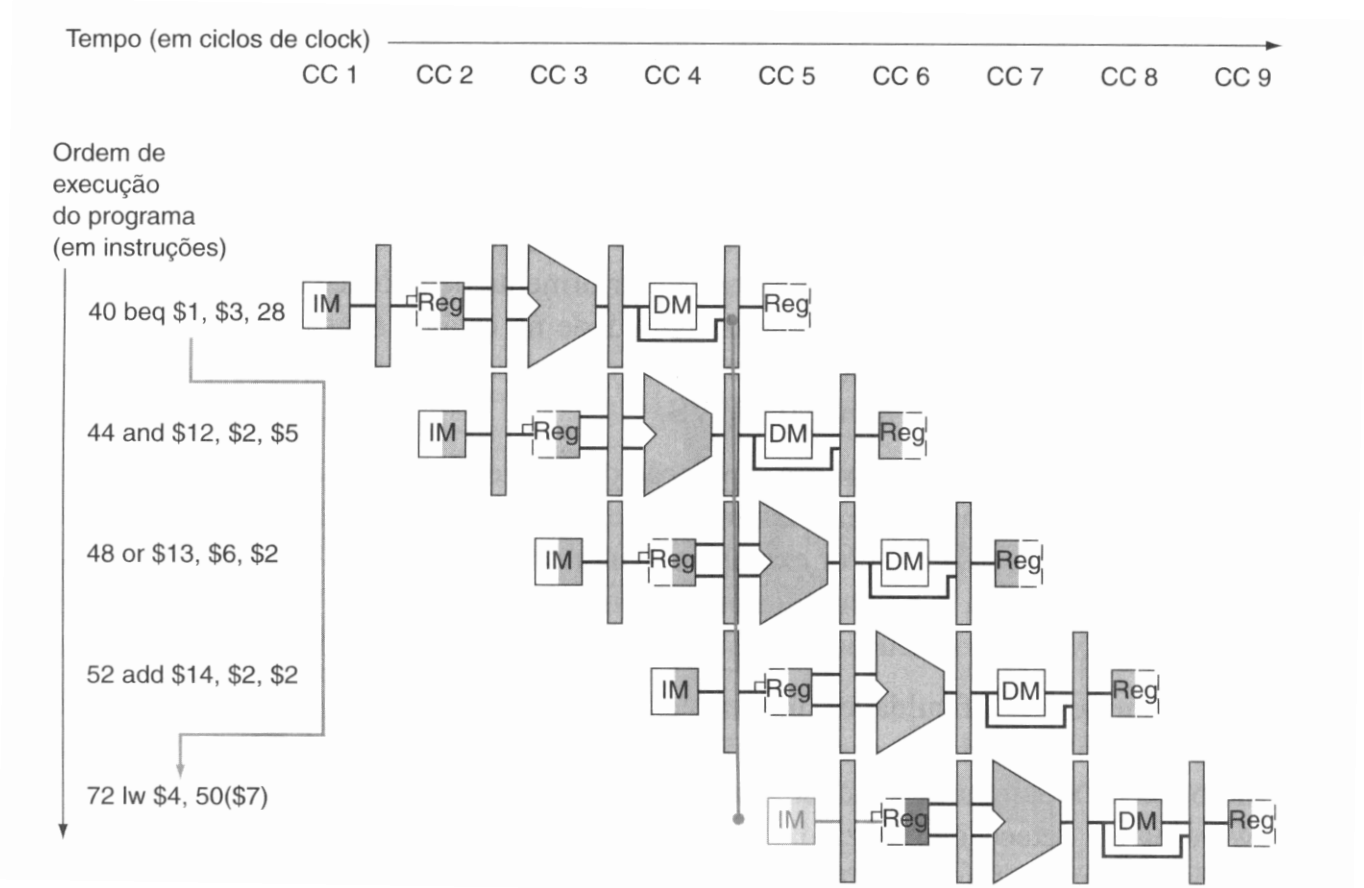
Roteiro

- Hazards de controle
- Reduzindo o atraso de desvios
- Previsão dinâmica de desvios
  - Buffer de 1 bit.
  - Buffer de 2 bits.

Hazards de controle

- Uma instrução precisa ser buscada a cada ciclo de clock para sustentar o pipeline
  - A decisão sobre o desvio não ocorrerá até o estágio MEM do pipeline.
  - O atraso para determinar a instrução apropriada a ser buscada é chamada de hazard de controle ou hazard de desvio
- Fazer um stall até que o desvio seja tomado.
  - Diminui o desempenho, pois é muito lento.
- Considerar que o desvio não será tomado
  - Continuar o fluxo normal de execução.
  - Se o desvio for tomado, as instruções que estão sendo buscadas e decodificadas são descartadas.
  - Para descartar as instruções nos estágios IF, ID e EX alteramos os valores de controle para 0 e descartamos as instruções no pipeline nos estágios IF, ID e EX (flush).

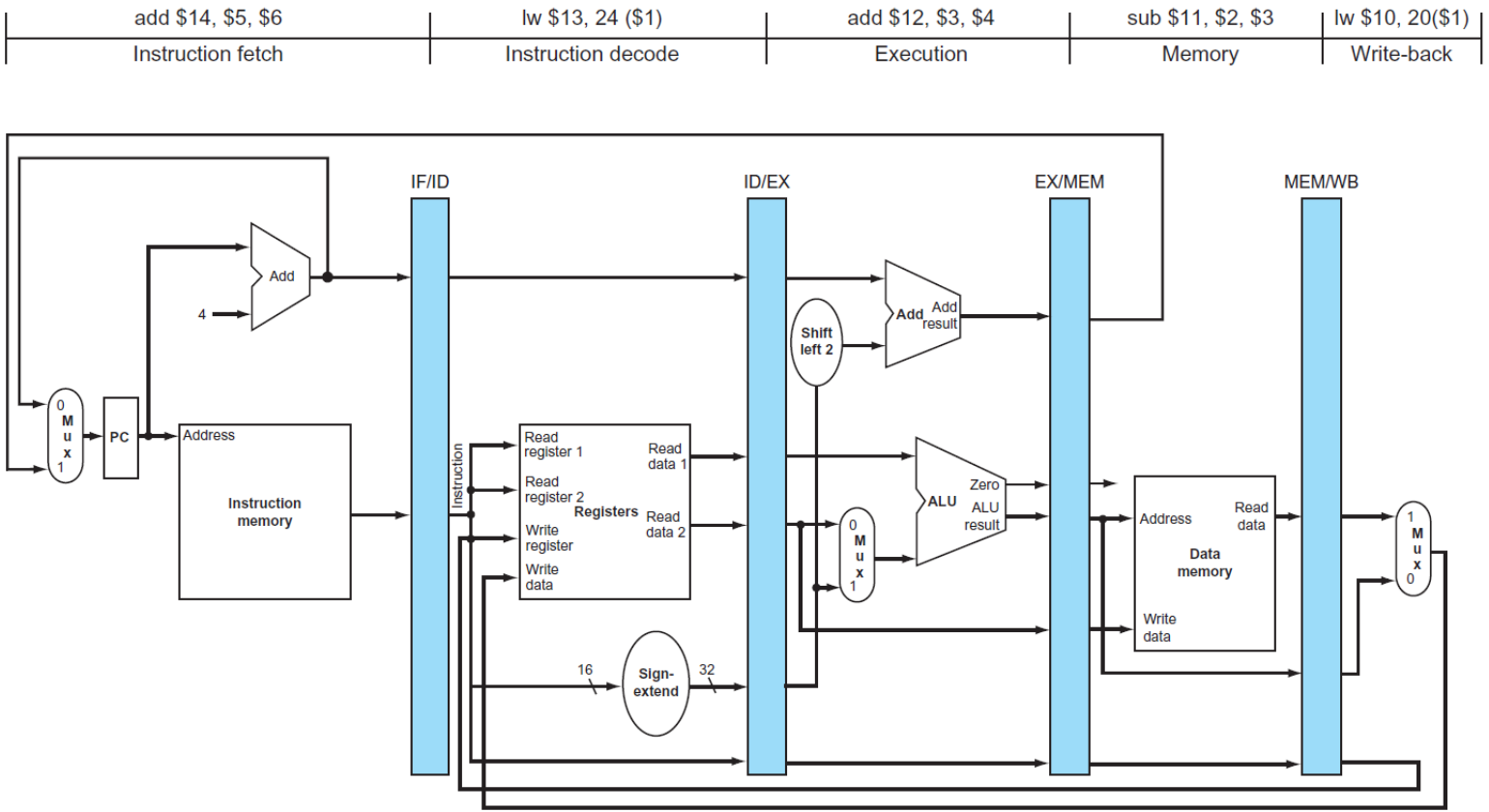
Hazards de controle



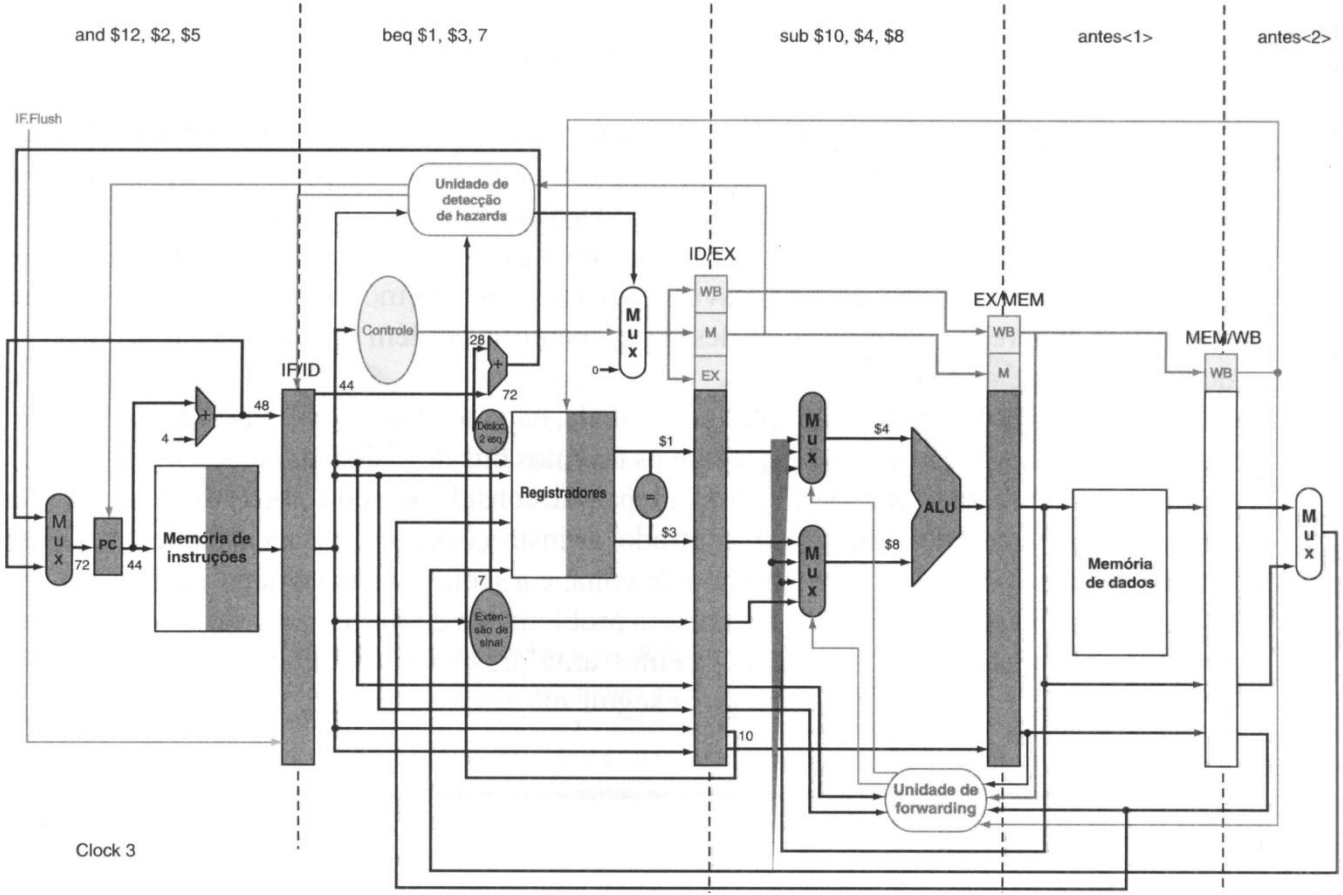
Reduzindo o atraso de desvios

- Muitos desvios contam apenas com testes simples:
  - Igualdade ou sinal;
- É possível reduzir o número de instruções que precisam sofrer flush, movendo a execução do desvio para um estágio anterior do pipeline.
  - Mover o calculo do desvio do estágio EX para o estágio ID:
    - Computar o endereço de desvio (Fácil)
      - Já temos o valor do PC + 4 e o endereço de destino em IF/ID
  - Decisão do desvio:
    - Comparar os dois registradores lidos no próprio estágio ID
    - Implica em hardware adicional para detecção de hazard e encaminhamento
      - Um dos valores testador pode não estar disponível no momento.

Reduzindo o atraso de desvios



Reduzindo o atraso de desvios

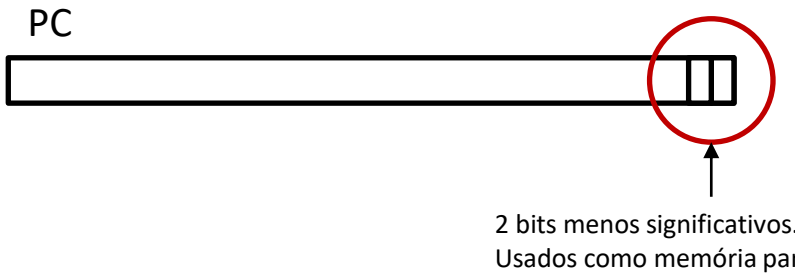


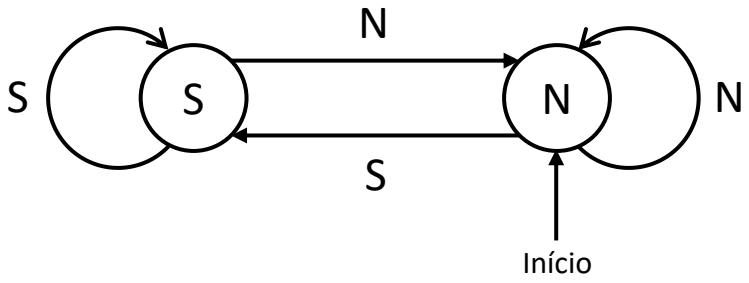
Previsão dinâmica de desvios

- A forma mais simples de previsão de desvios é supor que o desvio não é tomado.
  - Adequada para pipeline simples: Até cinco estágios (MIPS).
    - Em pipelines muito agressivos o desempenho desperdiçado será muito grande.
- Previsão dinâmica de desvios.
  - Decidir se o desvio será ou não tomado com base na ultima vez que a instrução foi executada
    - Buffer de previsão de desvios ou tabela de histórico de desvios
    - BHT – *buffer history table*

Previsão de desvio com 1 bit de história

- **Buffer de 1 bit:** O bit menos significativo do endereço do PC da instrução de desvio é utilizado como uma memória de desvios e diz se um desvio foi tomado recentemente
  - 1 – desvio será tomado
  - 0 – desvio não será tomado.
  - Mesmo que um desvio quase sempre seja tomado, provavelmente faremos uma previsão incorreta duas vezes.





Previsão de desvio com 2 bit de história

- Uma solução melhor é um esquema de previsão com 2 bits:
  - O bit MAIS significativo armazena a previsão
  - O bit MENOS significativo o resultado da última escolha.

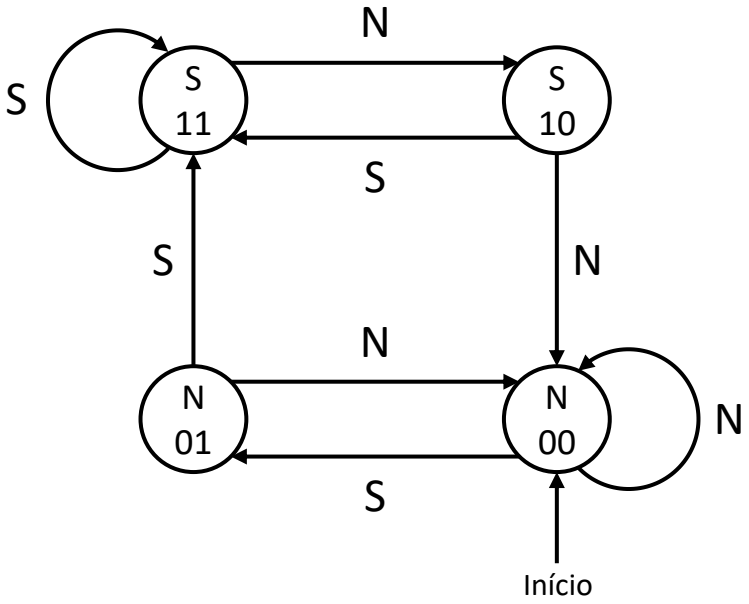
---

00 → Último branch **não foi** tomado.
Previsão: Próximo branch **não será** tomado.
(Estado estável)

01 → Último branch **foi** tomado.
Previsão: Próximo branch **não será** tomado.
(Estado de atenção)

11 → Último branch **foi** tomado.
Previsão: Próximo branch **será** tomado.
(Estado estável)

10 → Último branch **não foi** tomado.
Previsão: Próximo branch **será** tomado.
(Estado de atenção)



EXEMPLO

- Considere o código em assembly do MIPS:
  - Considere também que os registradores $s0, $s1 e $s2 armazenam os valores 0, 5 e 7, respectivamente.

```
Loop:  beq  $s0, $s1, Exit1
       addi $s0, $s0, 1
       j Loop
Exit1: bne $s0, $s1, Else
       add $s0, $s1, $s2
       j Exit
Else:  sub $s0, $s1, $s2
Exit:  # o código continua...
```

1 bit

P	N								
D									

2 bits

N									

EXEMPLO

- Considere o código em assembly do MIPS:
  - Considere também que os registradores $s0, $s1 e $s2 armazenam os valores 0, 5 e 7, respectivamente.

```
Loop:  beq  $s0, $s1, Exit1
       addi $s0, $s0, 1
       j Loop
Exit1: bne $s0, $s1, Else
       add $s0, $s1, $s2
       j Exit
Else:  sub $s0, $s1, $s2
Exit:  # o código continua...
```

$s0	$s1	$s0	$s1	$s0	$s1	$s0	$s1	$s0	$s1	$s0	$s1
0	5	1	5	2	5	3	5	4	5	5	5

$s0	$s1
5	5

1 bit

P	N	N	N	N	N	N	T	N		
D	N	N	N	N	N	T	N	...		
						X	X			

2 bits

N	N	N	N	N	N	N	N		
N	N	N	N	N	T	N	...		
					X	V			

Bibliografia

- PATTERSON, D. A.; HENNESSY, J. L. **Organização e Arquitetura de computadores: a interface hardware/software**. 3. ed. Rio de Janeiro: Elsevier, 2005.
  - CAPÍTULO 5
- Notas de aula da Profa. Mary Jane Irwin
  - CSE 331 Computer Organization and Design
  - http://www.cse.psu.edu/research/mdl/mji/mjicourses

FIM